

Tarea N° 04

Estructuras de Control en C y C++

Ejercicio 1: Suma de Números Pares

Enunciado

Escribir un programa que calcule la suma de todos los números pares desde 2 hasta N.

Código - Método 1 (Incremento de 2)

```
1 #include <stdio.h>
2
3 int main() {
4     int n, suma = 0;
5
6     printf("Ingrese N: ");
7     scanf("%d", &n);
8
9     for (int i = 2; i <= n; i += 2) {
10         suma += i;
11     }
12
13     printf("Suma: %d\n", suma);
14     return 0;
15 }
```

Salida

Ingrese N: 10

Suma: 30

Prueba de Escritorio

Entrada: N = 10

Iteración	i	$i \leq 10?$	suma
inicio	–	–	0
1	2	V	2
2	4	V	6
3	6	V	12
4	8	V	20
5	10	V	30
6	12	F	30

Salida: Suma: 30

Ejercicio 2: Conversión de Calificación

Enunciado

Crear un programa que convierta una nota numérica (0-20) a letra según el siguiente criterio:

- A (Excelente): 18-20
- B (Muy Bueno): 15-17
- C (Bueno): 11-14
- D (Regular): 8-10
- F (Desaprobado): 0-7

Código

```
1 #include <stdio.h>
2
3 int main() {
4     int nota;
5
6     printf("Ingrese la nota (0-20): ");
7     scanf("%d", &nota);
8
9     if (nota < 0 || nota > 20) {
10         printf("Error: Nota fuera de rango\n");
11     } else if (nota >= 18) {
12         printf("A - Excelente\n");
13     } else if (nota >= 15) {
14         printf("B - Muy Bueno\n");
15     } else if (nota >= 11) {
16         printf("C - Bueno\n");
17     } else if (nota >= 8) {
18         printf("D - Regular\n");
19     } else {
20         printf("F - Desaprobado\n");
21     }
22
23     return 0;
24 }
```

Salida

Ingrese la nota (0-20): 14

C - Bueno

Prueba de Escritorio

Nota	Rango?	≥18	≥15	≥11	≥8	Salida
19	V	V	–	–	–	A - Excelente
14	V	F	F	V	–	C - Bueno
8	V	F	F	F	V	D - Regular
5	V	F	F	F	F	F - Desaprobado
-3	F	–	–	–	–	Error

Ejercicio 3: Número Inverso

Enunciado

Escribir un programa que invierta un número entero positivo.

Código

```
1 #include <stdio.h>
2
3 int main() {
4     int n, original;
5     int inverso = 0;
6
7     printf("Ingrese numero: ");
8     scanf("%d", &n);
9     original = n;
10
11     while (n > 0) {
12         int digito = n % 10;
13         inverso = inverso * 10 + digito;
14         n = n / 10;
15     }
16
17     printf("Original: %d\n", original);
18     printf("Inverso: %d\n", inverso);
19
20     return 0;
21 }
```

Salida

Ingrese numero: 4831

Original: 4831

Inverso: 1384

Prueba de Escritorio

Entrada: n = 4831

Iter.	n	n > 0?	digito	cálculo	inverso
inicio	4831	—	—	—	0
1	4831	V	1	$0 \times 10 + 1$	1
	483				
2	483	V	3	$1 \times 10 + 3$	13
	48				
3	48	V	8	$13 \times 10 + 8$	138
	4				
4	4	V	4	$138 \times 10 + 4$	1384
	0				
5	0	F	—	—	1384

Algoritmo:

- $n \% 10$: obtiene el último dígito
- $\text{inverso} * 10 + \text{digito}$: agrega el dígito al inverso
- $n / 10$: elimina el último dígito

Ejercicio 4: Menú de Operaciones (Calculadora)

Enunciado

Crear una calculadora con menú que permita realizar operaciones básicas.

Código

```
1 #include <stdio.h>
2
3 int main() {
4     int opcion;
5     float a, b, resultado;
6
7     do {
8         printf("\n===== CALCULADORA =====\n");
9         printf("1. Suma\n");
10        printf("2. Resta\n");
11        printf("3. Multiplicacion\n");
12        printf("4. Division\n");
13        printf("0. Salir\n");
14        printf("Opcion: ");
15        scanf("%d", &opcion);
16
17        if (opcion >= 1 && opcion <= 4) {
18            printf("Numero 1: ");
19            scanf("%f", &a);
20            printf("Numero 2: ");
21            scanf("%f", &b);
22        }
23
24        switch (opcion) {
25            case 1:
26                printf("Resultado: %.2f\n", a + b);
27                break;
28            case 2:
29                printf("Resultado: %.2f\n", a - b);
30                break;
31            case 3:
32                printf("Resultado: %.2f\n", a * b);
33                break;
34            case 4:
35                if (b == 0)
36                    printf("Error: Division por cero!\n");
37                else
38                    printf("Resultado: %.2f\n", a / b);
39                break;
40            case 0:
41                printf("Saliendo...\n");
42                break;
43            default:
44                printf("Opcion no valida!\n");
45        }
```

```
46     } while (opcion != 0);  
47  
48     return 0;  
49 }
```

Salida

===== CALCULADORA =====

1. Suma
2. Resta
3. Multiplicacion
4. Division
0. Salir

Opcion: 1

Numero 1: 10

Numero 2: 5

Resultado: 15.00

===== CALCULADORA =====

...

Opcion: 4

Numero 1: 20

Numero 2: 0

Error: Division por cero!

Prueba de Escritorio

Iter.	Opción	a	b	Operación	Resultado
1	1 (Suma)	10	5	$10 + 5$	15.00
2	3 (Mult)	4	7	4×7	28.00
3	4 (Div)	20	0	$20 \div 0$	Error
4	4 (Div)	20	4	$20 \div 4$	5.00
5	2 (Resta)	15	8	$15 - 8$	7.00
6	5	—	—	—	Opción no válida
7	0 (Salir)	—	—	—	Saliendo...

Estructuras utilizadas:

- do-while: para el menú repetitivo
- switch-case: para seleccionar operaciones
- if: para validar división por cero

Ejercicio 5: Patrones de Asteriscos

Patrón A: Triángulo Ascendente

Código

```
1 #include <stdio.h>
2
3 int main() {
4     int n;
5
6     printf("Ingrese N: ");
7     scanf("%d", &n);
8
9     for (int i = 1; i <= n; i++) {
10         for (int j = 1; j <= i; j++) {
11             printf("*");
12         }
13         printf("\n");
14     }
15
16     return 0;
17 }
```

Salida (N=4)

```
*
**
***
****
```

Prueba de Escritorio

i	j (1 a i)	Salida
1	1	*
2	1, 2	**
3	1, 2, 3	***
4	1, 2, 3, 4	****

Patrón B: Triángulo Descendente

Código

```
1 #include <stdio.h>
2
3 int main() {
4     int n;
5
6     printf("Ingrese N: ");
7     scanf("%d", &n);
8
9     for (int i = n; i >= 1; i--) {
10         for (int j = 1; j <= i; j++) {
11             printf("*");
12         }
13         printf("\n");
14     }
15
16     return 0;
17 }
```

Salida (N=4)

```
****
***
**
*
```

Prueba de Escritorio

i	j (1 a i)	Salida
4	1, 2, 3, 4	****
3	1, 2, 3	***
2	1, 2	**
1	1	*

Patrón C: Pirámide

Código

```

1 #include <stdio.h>
2
3 int main() {
4     int n;
5
6     printf("Ingrese N: ");
7     scanf("%d", &n);
8
9     for (int i = 1; i <= n; i++) {
10         // Espacios
11         for (int e = 1; e <= n - i; e++) {
12             printf(" ");
13         }
14         // Asteriscos
15         for (int a = 1; a <= 2*i - 1; a++) {
16             printf("*");
17         }
18         printf("\n");
19     }
20
21     return 0;
22 }

```

Salida (N=4)

```

    *
  ***
 *****
*****

```

Prueba de Escritorio

i	Espacios (n-i)	Asteriscos (2i-1)	Salida
1	3	1	*
2	2	3	***
3	1	5	*****
4	0	7	*****

Fórmulas:

- $\text{Espacios} = n - i$
- $\text{Asteriscos} = 2i - 1$

Ejercicio 6: Número Perfecto

Enunciado

Un número perfecto es aquel que es igual a la suma de sus divisores propios (excluyendo el mismo número). Ejemplo: 6 es perfecto porque $1 + 2 + 3 = 6$.

Código

```
1 #include <stdio.h>
2
3 int main() {
4     int n, suma = 0;
5
6     printf("Ingrese un numero: ");
7     scanf("%d", &n);
8
9     if (n <= 1) {
10         printf("Error: El numero debe ser mayor que 1\n");
11         return 1;
12     }
13
14     printf("Divisores de %d: ", n);
15
16     for (int i = 1; i < n; i++) {
17         if (n % i == 0) {
18             printf("%d ", i);
19             suma += i;
20         }
21     }
22
23     printf("\nSuma de divisores: %d\n", suma);
24
25     if (suma == n) {
26         printf("%d ES un numero perfecto\n", n);
27     } else {
28         printf("%d NO es un numero perfecto\n", n);
29     }
30
31     return 0;
32 }
```

Salida

```
Ingrese un numero: 28
Divisores de 28: 1 2 4 7 14
Suma de divisores: 28
28 ES un numero perfecto
```

Prueba de Escritorio

Entrada: $n = 28$

i	$28 \% i == 0?$	Divisor	suma
1	V	1	1
2	V	2	3
3	F	—	3
4	V	4	7
5	F	—	7
6	F	—	7
7	V	7	14
8-13	F	—	14
14	V	14	28
15-27	F	—	28

Resultado: $\text{suma} = 28 = n \Rightarrow 28$ ES perfecto

Caso: $n = 12$

i	$12 \% i == 0?$	Divisor	suma
1	V	1	1
2	V	2	3
3	V	3	6
4	V	4	10
5	F	—	10
6	V	6	16
7-11	F	—	16

Resultado: $\text{suma} = 16 \neq 12 \Rightarrow 12$ NO es perfecto

Ejercicio 7: Serie de Fibonacci

Enunciado

La serie de Fibonacci es una secuencia donde cada número es la suma de los dos anteriores, comenzando con 0 y 1. Serie: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34...

Fórmula: $F(n) = F(n - 1) + F(n - 2)$

Código

```
1 #include <stdio.h>
2
3 int main() {
4     int n, a = 0, b = 1, siguiente;
5
6     printf("Ingrese cuantos terminos: ");
7     scanf("%d", &n);
8
9     if (n <= 0) {
10         printf("Error: N debe ser positivo\n");
11         return 1;
12     }
13
14     printf("Serie de Fibonacci (%d terminos):\n", n);
15
16     for (int i = 1; i <= n; i++) {
17         if (i == 1) {
18             printf("%d ", a);
19             continue;
20         }
21         if (i == 2) {
22             printf("%d ", b);
23             continue;
24         }
25         siguiente = a + b;
26         printf("%d ", siguiente);
27         a = b;
28         b = siguiente;
29     }
30
31     printf("\n");
32     return 0;
33 }
```

Salida

```
Ingrese cuantos terminos: 8
Serie de Fibonacci (8 terminos):
0 1 1 2 3 5 8 13
```

Prueba de Escritorio

Entrada: $n = 8$

i	a	b	siguiente	Imprime	Nueva a, b
inicio	0	1	—	—	—
1	0	1	—	0	0, 1
2	0	1	—	1	0, 1
3	0	1	$0 + 1 = 1$	1	1, 1
4	1	1	$1 + 1 = 2$	2	1, 2
5	1	2	$1 + 2 = 3$	3	2, 3
6	2	3	$2 + 3 = 5$	5	3, 5
7	3	5	$3 + 5 = 8$	8	5, 8
8	5	8	$5 + 8 = 13$	13	8, 13

Salida: 0 1 1 2 3 5 8 13

Casos especiales:

- $N = 1$: Imprime solo “0”
- $N = 2$: Imprime “0 1”
- $N = 3$: Imprime “0 1 1”

Ejercicio 8: Validador de Contraseña

Enunciado

Crear un programa que valide si una contraseña cumple criterios de seguridad:

1. Longitud mínima de 8 caracteres
2. Al menos una letra mayúscula (A-Z)
3. Al menos una letra minúscula (a-z)
4. Al menos un dígito (0-9)
5. Al menos un carácter especial (!, @, #, \$, %)

Código

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char password[100];
6     int longitud, tiene_mayus = 0, tiene_minus = 0;
7     int tiene_digito = 0, tiene_especial = 0;
8
9     printf("Ingrese contrasena: ");
10    scanf("%s", password);
11
12    longitud = strlen(password);
13
14    // Verificar cada caracter
15    for (int i = 0; i < longitud; i++) {
16        char c = password[i];
17
18        if (c >= 'A' && c <= 'Z')
19            tiene_mayus = 1;
20        else if (c >= 'a' && c <= 'z')
21            tiene_minus = 1;
22        else if (c >= '0' && c <= '9')
23            tiene_digito = 1;
24        else if (c == '!' || c == '@' || c == '#' ||
25                c == '$' || c == '%')
26            tiene_especial = 1;
27    }
28
29    // Mostrar resultados
30    printf("\n--- Analisis de Contraseña ---\n");
31    printf("Longitud: %d %s\n", longitud,
32           longitud >= 8 ? "[OK]" : "[FALTA]");
33    printf("Mayuscula: %s\n", tiene_mayus ? "[OK]" : "[FALTA]");
34    printf("Minuscula: %s\n", tiene_minus ? "[OK]" : "[FALTA]");
35    printf("Digito: %s\n", tiene_digito ? "[OK]" : "[FALTA]");
36    printf("Especial: %s\n", tiene_especial ? "[OK]" : "[FALTA]");
```



```
37
38     if (longitud >= 8 && tiene_mayus && tiene_minus &&
39         tiene_digito && tiene_especial) {
40         printf("\nContraseña VALIDA\n");
41     } else {
42         printf("\nContraseña INVALIDA\n");
43     }
44
45     return 0;
46 }
```

Salida

Ingrese contrasena: Hola123!

--- Analisis de Contraseña ---

Longitud: 8 [OK]

Mayuscula: [OK]

Minuscula: [OK]

Digito: [OK]

Especial: [OK]

Contraseña VALIDA

Prueba de Escritorio

Entrada: password = "Hola123!"

i	c	mayus	minus	digito	especial
inicio	—	0	0	0	0
0	'H'	1	0	0	0
1	'o'	1	1	0	0
2	'l'	1	1	0	0
3	'a'	1	1	0	0
4	'1'	1	1	1	0
5	'2'	1	1	1	0
6	'3'	1	1	1	0
7	'!'	1	1	1	1

Evaluación final:

- Longitud = $8 \geq 8$
- tiene_mayus = 1
- tiene_minus = 1
- tiene_digito = 1
- tiene_especial = 1

Resultado: Contraseña VALIDA

Caso inválido: "hola123"

- Longitud = $7 < 8$ ×
- Sin mayúsculas ×
- Sin caracteres especiales ×
- **Resultado:** INVALIDA

Ejercicio 9: Conversión de Bases Numéricas

Enunciado

Escribir un programa que convierta un número decimal a otras bases (binario, octal, hexadecimal).

Código

```
1 #include <stdio.h>
2
3 void decimalABinario(int n) {
4     int binario[32], i = 0;
5
6     if (n == 0) {
7         printf("0");
8         return;
9     }
10
11     while (n > 0) {
12         binario[i] = n % 2;
13         n = n / 2;
14         i++;
15     }
16
17     for (int j = i - 1; j >= 0; j--)
18         printf("%d", binario[j]);
19 }
20
21 void decimalAOctal(int n) {
22     int octal[32], i = 0;
23
24     if (n == 0) {
25         printf("0");
26         return;
27     }
28
29     while (n > 0) {
30         octal[i] = n % 8;
31         n = n / 8;
32         i++;
33     }
34
35     for (int j = i - 1; j >= 0; j--)
36         printf("%d", octal[j]);
37 }
38
39 void decimalAHexadecimal(int n) {
40     char hex[32];
41     int i = 0;
42
43     if (n == 0) {
44         printf("0");
```

```
45     return;
46 }
47
48 while (n > 0) {
49     int residuo = n % 16;
50     if (residuo < 10)
51         hex[i] = '0' + residuo;
52     else
53         hex[i] = 'A' + (residuo - 10);
54     n = n / 16;
55     i++;
56 }
57
58 for (int j = i - 1; j >= 0; j--)
59     printf("%c", hex[j]);
60 }
```

```
1 int main() {
2     int opcion, numero;
3
4     do {
5         printf("\n=== CONVERSION DE BASES ===\n");
6         printf("1. Decimal a Binario\n");
7         printf("2. Decimal a Octal\n");
8         printf("3. Decimal a Hexadecimal\n");
9         printf("0. Salir\n");
10        printf("Opcion: ");
11        scanf("%d", &opcion);
12
13        if (opcion >= 1 && opcion <= 3) {
14            printf("Ingrese numero decimal: ");
15            scanf("%d", &numero);
16
17            if (numero < 0) {
18                printf("Error: Solo numeros positivos\n");
19                continue;
20            }
21
22            printf("Resultado: ");
23
24            switch (opcion) {
25                case 1:
26                    decimalABinario(numero);
27                    printf(" (binario)\n");
28                    break;
29                case 2:
30                    decimalAOctal(numero);
31                    printf(" (octal)\n");
32                    break;
33                case 3:
34                    decimalAHexadecimal(numero);
35                    printf(" (hexadecimal)\n");
36                    break;
37            }
38        }
39        while (opcion != 0);
40
41        printf("Saliendo...\n");
42        return 0;
43 }
```

Salida

=== CONVERSION DE BASES ===

1. Decimal a Binario
2. Decimal a Octal
3. Decimal a Hexadecimal
0. Salir
Opcion: 1

Ingrese numero decimal: 25

Resultado: 11001 (binario)

Opcion: 3

Ingrese numero decimal: 255

Resultado: FF (hexadecimal)

Prueba de Escritorio - Decimal a Binario

Entrada: numero = 25

Iter.	n	n % 2	n / 2	binario[i]
inicio	25	—	—	—
1	25	1	12	binario[0] = 1
2	12	0	6	binario[1] = 0
3	6	0	3	binario[2] = 0
4	3	1	1	binario[3] = 1
5	1	1	0	binario[4] = 1
6	0	—	—	i = 5

Array binario: [1, 0, 0, 1, 1] (índices 0-4)

Imprimir al revés: binario[4], binario[3], binario[2], binario[1], binario[0]

Resultado: 11001

Verificación: $1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 16 + 8 + 1 = 25$

Prueba de Escritorio - Decimal a Hexadecimal

Entrada: numero = 255

Iter.	n	n % 16	Conversión	hex[i]
inicio	255	—	—	—
1	255	15	15 → 'F'	hex[0] = 'F'
2	15	15	15 → 'F'	hex[1] = 'F'
3	0	—	—	i = 2

Array hex: ['F', 'F']

Resultado: FF

Verificación: $15 \times 16^1 + 15 \times 16^0 = 240 + 15 = 255$

Ejercicio 10: Juego de Adivinanza

Enunciado

Crear un juego donde la computadora genera un número aleatorio entre 1 y 100, y el usuario debe adivinarlo con pistas.

Código

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 int main() {
6     int numeroSecreto, intento, contador = 0;
7     int MAX_INTENTOS = 10;
8
9     // Inicializar generador de numeros aleatorios
10    srand(time(0));
11    numeroSecreto = rand() % 100 + 1; // Numero entre 1 y 100
12
13    printf("=== JUEGO DE ADIVINANZA ===\n");
14    printf("Adivina el numero (1-100)\n");
15    printf("Tienes %d intentos!\n\n", MAX_INTENTOS);
16
17    do {
18        contador++;
19        printf("Intento %d: ", contador);
20        scanf("%d", &intento);
21
22        if (intento < 1 || intento > 100) {
23            printf("Por favor, ingresa un numero entre 1 y 100\n");
24            contador--;
25            continue;
26        }
27
28        if (intento < numeroSecreto) {
29            printf("-> El numero es MAYOR\n\n");
30        } else if (intento > numeroSecreto) {
31            printf("-> El numero es MENOR\n\n");
32        } else {
33            printf("-> FELICIDADES! Adivinaste en %d intentos!\n",
34                contador);
35            return 0;
36        }
37
38        if (contador >= MAX_INTENTOS) {
39            printf("\nLo siento, alcanzaste el limite de intentos.\n");
40            ;
41            printf("El numero era: %d\n", numeroSecreto);
42            return 0;
43        }
44    }
```



```
44     } while (intento != numeroSecreto);  
45  
46     return 0;  
47 }
```

Salida

=== JUEGO DE ADIVINANZA ===

Adivina el numero (1-100)

Tienes 10 intentos!

Intento 1: 50

-> El numero es MAYOR

Intento 2: 75

-> El numero es MENOR

Intento 3: 62

-> El numero es MAYOR

Intento 4: 68

-> FELICIDADES! Adivinaste en 4 intentos!

Prueba de Escritorio

Número secreto generado: 68

contador	intento	¿68?	¿68?	Mensaje
1	50	V	F	El número es MAYOR
2	75	F	V	El número es MENOR
3	62	V	F	El número es MAYOR
4	68	F	F	FELICIDADES (4 intentos)

Algoritmo de búsqueda del usuario:

1. Inicia en 50 (punto medio de 1-100)
2. Es mayor → busca entre 51-100
3. Prueba 75
4. Es menor → busca entre 51-74
5. Prueba 62
6. Es mayor → busca entre 63-74
7. Prueba 68

Caso: Alcanza máximo de intentos

Después de 10 intentos sin acertar:

Lo siento, alcanzaste el limite de intentos.

El numero era: 68

Funciones utilizadas:

- `srand(time(0))`: inicializa el generador con semilla basada en tiempo
- `rand() % 100 + 1`: genera número aleatorio entre 1 y 100
- `time(0)`: obtiene tiempo actual en segundos